Module 5 [Advanced Topics]:

Topic 1[Exception Handling]

Problem Statement: In the following input `i` is the index of array `ax[].` The program prints `ax[i].` Then write catch block if i is out of range of `ax[].` Write three `catch` blocks to fullfill the purpose

i) a `catch` block receives the value of `i`
ii) a `catch` block receives string "`Out of Range Error`"
iii) a default `catch()` if above two `catch` block doesn't match

```
#include <iostream>
using namespace std;

int main()
{
  int i;
  int ax[5]={10,20,60,40,30};
  cout<<"enter index:";
  cin>>i;
  cout<<"ax["<<i<<"]="<<ax[i]<<endl;
}
```

Code:

```
#include <iostream>
using namespace std;

int main()
{
  int i;
  int ax[5]={10,20,60,40,30};
  cout<<"Enter index(int):";
  cin>>i;
  try
  {
    if(i>4 || i<0)
    {
        throw(i);
    }
    if(cin.fail())
    {
        throw("notinteger");
    }
    cout<<"ax["<<i<<"]="<<ax[i]<<endl;
  }
  catch(int i)
{
    cout << "Out of Range" << endl;
}
  catch(const char *s)
```

```
  {
    cout << s << endl;
  }
  catch(...)
  {
    cout << "Exception is exception" << endl;
  }
  return 0;
}
```

Topic 2 [**class Template**]:

Problem Statement: In the following class data members `x` and `y` are integers and the method `Sum()` adds `x` and `y`. However, we need to perform the sum of `int+int, int+double, douuble+int` and `double+double` . To achieve it, change the definition of `x,y,setData()` and `Sum()` accordingly.

```
class A{
 private:
    int x;
    int y;
  public:
    void setData(int x,int y){
     this->x=x;
     this->y=y;
    }
    int Sum(){
      int s;
      s=x+y;
      return s;
    }
};
int main(){
 //write required statements to call SetData() & Sum()
}
```

Code:

```
#include <iostream>
using namespace std;
template <class T1, class T2>
class A
{
```

```
    T1 x;
    T2 y;
public:
    void setData(T1 x, T2 y) {
       this-> x=x;
        this ->y =y;
    }
    auto Sum()
    {
        return x + y;
    }
};
int main()
{

    A<int, int> obj1;
    obj1.setData(10, 20);
    cout << "Sum(int, int): " << obj1.Sum() << endl;
    A<int, double> obj2;
    obj2.setData(10, 5.5);
    cout << "Sum(int, double): " << obj2.Sum() << endl;
    A<double, int> obj3;
    obj3.setData(3.5, 7);
    cout << "Sum(double, int): " << obj3.Sum() << endl;
    A<double, double> obj4;
    obj4.setData(2.5, 3.5);
    cout << "Sum(double, double): " << obj4.Sum() << endl;
    return 0;
}
```

Topic 3 [STL:Array class]

Problem statement: Declare a STL array object ax with 6 elements and do the following:

i) Assign 10,60,30,70,20 to ax using single statement
ii) Print third element of ax using at() function
iii) Print first element of ax using front() function
iv) Print last element of ax using back() function
v) Fill the elements of ax using fill() function
vi) Test whether ax is empty or not using empty() function
vii) Print size of ax
viii) Print maximum size of ax using max_size() function
ix) Print address of first element of ax using begin() function
x) Print address of last element of ax using end() function

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
int main(){
  array<int,6>ax;
  //write statements
}
```

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    array<int, 6> ax;

    // i) assign using single statement
    ax = {10, 60, 30, 70, 20, 90};

    // ii) third element
    cout << "Third element: " << ax.at(2) << endl;

    // iii) first element
    cout << "First element: " << ax.front() << endl;

    // iv) last element
    cout << "Last element: " << ax.back() << endl;

    // v) fill
    ax.fill(5);
    cout << "After fill(): ";
    for (int x : ax) cout << x << " ";
    cout << endl;
```

```
    // vi) empty
    cout << "Is empty: " << (ax.empty() ? "Yes" : "No") << endl;

    // vii) size
    cout << "Size: " << ax.size() << endl;

    // viii) max_size
    cout << "Max size: " << ax.max_size() << endl;

    // ix) address of first element
    cout << "Address of first element: " << &(*ax.begin()) << endl;

    // x) address of last element
    cout << "Address of last element: " << &(*(ax.end() - 1)) << endl;

    return 0;
}
```

```
PS D:\Satu\1-2\CSE 1204\week 5> cd "d:\s
g++ stlarray.cpp -o stlarray } ; if ($?)
Third element: 30
First element: 10
Last element: 90
After fill(): 5 5 5 5 5 5
Is empty: No
Size: 6
Max size: 6
Address of first element: 0x61fee4
Address of last element: 0x61fef8
```

Topic 4[STL: pair class]

Problem statement: Define a pair class object px with int and string elements. Write statements to do the following

i) Assign 10 to int and “Rajshahi” to px using make_pair() function
ii) Print int data member by first
iii) Print string data member by second
iv) Modify first data member to 20 using get<>() function
v) Declare another pair bx and assign values to bx and swap it with ax

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
int main(){
  pair<int,string>px;
  //write statements
}
```

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    pair<int, string> px;

    // i)
    px = make_pair(10, "Rajshahi");

    // ii)
    cout << "First: " << px.first << endl;

    // iii)
    cout << "Second: " << px.second << endl;

    // iv)
    get<0>(px) = 20;
    cout << "Modified First: " << px.first << endl;

    // v)
    pair<int, string> bx;
    bx = make_pair(50, "Dhaka");

    px.swap(bx);

    cout << "After swap, px = (" << px.first << ", " << px.second << ")" << endl;
    cout << "After swap, bx = (" << bx.first << ", " << bx.second << ")" << endl;

    return 0;
}
```

```
PS D:\Satu\1-2\CSE 1204\week 5>
g++ stlpair.cpp -o stlpair } ;
First: 10
Second: Rajshahi
Modified First: 20
After swap, px = (50, Dhaka)
After swap, bx = (20, Rajshahi)
```

Topic 5 [STL: tuple class]

iii) Print string data member by get() function
iv) Print double data member by get() function
v) Modify third data member to 3.7 using get<>() function
vi) Declare another tuple bx and assign values to bx and swap it with ax

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
int main(){
  tuple<int,string,double>tx;
  //write statements
}
```

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    tuple<int, string, double> tx;

    // i)
    tx = make_tuple(100, "Kamal", 3.5);

    // ii)
    cout << "Int: " << get<0>(tx) << endl;

    // iii)
    cout << "String: " << get<1>(tx) << endl;

    // iv)
    cout << "Double: " << get<2>(tx) << endl;

    // v)
    get<2>(tx) = 3.7;
    cout << "Modified Double: " << get<2>(tx) << endl;

    // vi)
    tuple<int, string, double> bx;
    bx = make_tuple(200, "Rahim", 4.2);

    tx.swap(bx);

    cout << "After swap, tx = ("
         << get<0>(tx) << ", "
         << get<1>(tx) << ", "
         << get<2>(tx) << ")" << endl;

    cout << "After swap, bx = ("
```

```
        << get<0>(bx) << ", "
        << get<1>(bx) << ", "
        << get<2>(bx) << ")" << endl;

    return 0;
}
```

```
PS D:\Satu\1-2\CSE 1204\week 5> 
g++ stlpair.cpp -o stlpair } ; i
First: 10
Second: Rajshahi
Modified First: 20
After swap, px = (50, Dhaka)
After swap, bx = (20, Rajshahi)
```

Topic 6 [STL: stack class]

Problem statement: Write a program to create and manipulate Stack using `stack` class and Perform the following operations using the specified method.

i) use `push()` method to push data
ii) use `pop()` method to pop data
iii) use `top()` method to display top element
iv) use `empty()` method to check whether stack is empty or not

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
int main(){
  stack<int>st;
  //write statements
}
```

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st;

    // push()
    st.push(10);
    st.push(20);
    st.push(30);

    // top()
    cout << "Top element: " << st.top() << endl;

    // pop()
    st.pop();
    cout << "After pop, top element: " << st.top() << endl;

    // empty()
    if (st.empty())
        cout << "Stack is empty" << endl;
    else
        cout << "Stack is not empty" << endl;

    return 0;
}
```

```
PS D:\Satu\1-2\CSE 1204\wee
g++ stackclass.cpp -o stack
Top element: 30
After pop, top element: 20
Stack is not empty
```

Topic 7 [STL: queue class]

Problem statement: Write a program to create and manipulate Queue using queue

class. Perform the following operations using the specified method.

i) use `push()` method to push data
ii) use `pop()` method to pop data
iii) use `front()` method to display front element
iv) use `back()` method to display rear element
v) use `empty()` method to check whether queue is empty or not

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
int main(){
  queue<int>qu;
```

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    queue<int> qu;

    // push()
    qu.push(10);
    qu.push(20);
    qu.push(30);

    // front()
    cout << "Front element: " << qu.front() << endl;

    // back()
    cout << "Rear element: " << qu.back() << endl;

    // pop()
    qu.pop();
    cout << "After pop, front element: " << qu.front() << endl;

    // empty()
    if (qu.empty())
        cout << "Queue is empty" << endl;
    else
        cout << "Queue is not empty" << endl;

    return 0;
}
```

```
  //write statements
}
```

Topic 8 [STL: list class]

Problem statement: Write a program to create and manipulate linked list using `list` class and its following methods to

i) insert 8 integers using **`push_back()`** method
ii) insert two elements using **`push_front()`** method
iii) display all the elements of the list in forward direction with user-defined Display() method using **`begin()`** and **`end()`** methods and iterator
iv) display all the elements of the list in reverse direction with a user-defined DisplayRev() method using **`rbegin()`** and rend() method and iterator
v) display front element using **`front()`** method
vi) display back element using **`back()`** method
vii) delete front element using **`pop_back()`** method

viii) delete front element using **pop_front()** method
ix) search an element **x** using **find()** method
x) insert a new element **x** before an existing element **y** using **insert()** method
xi) insert a new element **x** after an existing element **y** using **insert()** method
xii) count a particular element **x**
xiii) count a elements with condition using predicate function
xiv) delete a particular element **x** with **erase()** method
xv) delete first 4 elements with **erase()** method
xvi) delete a particular element **x** with **remove()** method
xvii) delete a elements with condition using **remove_if()** method using predicate function
xviii) assign elements from another list using **assign()** method
xix) assign elements from an array using **assign()** method
xx) sort the list using **sort()** method
xxi) Delete consecutive similar elements using **unique()** method

Code:

```
#include <iostream>
#include <bits/stdc++.h>
#include <iterator>
#include <algorithm>
using namespace std;

int main(){
  list<int>li;
  //write statements
}
```

```
#include <bits/stdc++.h>
#include <iterator>
#include <algorithm>
using namespace std;

bool isEven(int x) {
    return x % 2 == 0;
}

void Display(list<int> li) {
    list<int>::iterator it;
    for (it = li.begin(); it != li.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

void DisplayRev(list<int> li) {
    list<int>::reverse_iterator it;
```

```
    for (it = li.rbegin(); it != li.rend(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

int main() {
    list<int> li;

    // i) insert 8 integers using push_back()
    li.push_back(10);
    li.push_back(20);
    li.push_back(30);
    li.push_back(40);
    li.push_back(50);
    li.push_back(60);
    li.push_back(70);
    li.push_back(80);

    // ii) insert two using push_front()
    li.push_front(5);
    li.push_front(1);

    cout << "Forward: ";
    Display(li);

    // iv) reverse display
    cout << "Reverse: ";
    DisplayRev(li);

    // v) front()
    cout << "Front element: " << li.front() << endl;

    // vi) back()
    cout << "Back element: " << li.back() << endl;

    // vii) delete back using pop_back()
    li.pop_back();
    cout << "After pop_back: ";
    Display(li);

    // viii) delete front using pop_front()
    li.pop_front();
    cout << "After pop_front: ";
    Display(li);

    // ix) search an element x using find()
    int x = 30;
    auto it = find(li.begin(), li.end(), x);
```

```
    if (it != li.end())
        cout << x << " found" << endl;
    else
        cout << x << " not found" << endl;

    // x) insert a new element x before existing y
    int newVal1 = 25, y1 = 30;
    auto it1 = find(li.begin(), li.end(), y1);
    if (it1 != li.end()) {
        li.insert(it1, newVal1);
    }
    cout << "After insert before 30: ";
    Display(li);

    // xi) insert a new element x after existing y
    int newVal2 = 35, y2 = 30;
    auto it2 = find(li.begin(), li.end(), y2);
    if (it2 != li.end()) {
        ++it2;
        li.insert(it2, newVal2);
    }
    cout << "After insert after 30: ";
    Display(li);

    // xii) count a particular element x
    cout << "Count of 30: " << count(li.begin(), li.end(), 30) << endl;

    // xiii) count elements with condition
    cout << "Count of even elements: " << count_if(li.begin(), li.end(), isEven) <<
endl;

    // xiv) delete a particular element x with erase()
    auto it3 = find(li.begin(), li.end(), 25);
    if (it3 != li.end()) {
        li.erase(it3);
    }
    cout << "After erase(25): ";
    Display(li);

    // xv) delete first 4 elements with erase()
    auto start = li.begin();
    auto finish = li.begin();
    advance(finish, 4);
    li.erase(start, finish);
    cout << "After deleting first 4 elements: ";
    Display(li);

    // xvi) delete a particular element x with remove()
    li.remove(60);
```

```
    cout << "After remove(60): ";
    Display(li);

    // xvii) delete elements with condition using remove_if()
    li.remove_if(isEven);
    cout << "After remove_if(isEven): ";
    Display(li);

    // xviii) assign elements from another list
    list<int> li2;
    li2.push_back(100);
    li2.push_back(200);
    li2.push_back(300);

    li.assign(li2.begin(), li2.end());
    cout << "After assign from another list: ";
    Display(li);

    // xix) assign elements from an array
    int arr[] = {7, 8, 9, 10};
    li.assign(arr, arr + 4);
    cout << "After assign from array: ";
    Display(li);

    // xx) sort the list
    li.push_back(2);
    li.push_back(15);
    li.sort();
    cout << "After sort: ";
    Display(li);

    // xxi) delete consecutive similar elements using unique()
    li.push_back(15);
    li.push_back(15);
    li.push_back(20);
    li.push_back(20);
    li.sort();
    cout << "Before unique: ";
    Display(li);

    li.unique();
    cout << "After unique: ";
    Display(li);

    return 0;
}
```

```
After erase(25): 5 10 20 30 35 40 50 60 70
After deleting first 4 elements: 35 40 50 60 70
After remove(60): 35 40 50 70
After remove_if(isEven): 35
After assign from another list: 100 200 300
After assign from array: 7 8 9 10
After sort: 2 7 8 9 10 15
Before unique: 2 7 8 9 10 15 15 15 20 20
After unique: 2 7 8 9 10 15 20 _
```